

Análisis del sistema — qué anda mal y qué se puede mejorar

Alcance: backend `gestion_rrhh/` (Django + DRF + Postgres) y frontend `frontend/` (canvas de Claude Design + capa de integración `ceibo-api.js` + `build.py`). **Fecha:** 2026-07-14 · rama `fase-0-verificada`. **Última actualización:** 2026-07-16 — las secciones **B (correctitud e integridad)** y **C (robustez y rendimiento)** están **resueltas**; sus hallazgos se quitaron de este documento (ver "B. Correctitud..." y "C. Robustez..." abajo). De la sección D se cerraron **D6, D7 y D8**, y se corrigieron **D3 y D4**, que ya estaban hechos cuando se escribió este análisis: el relevamiento los dio por pendientes de más. **Toda la sección A (seguridad) sigue abierta.**

Cada hallazgo tiene un ID (A = seguridad, B = correctitud/integridad, C = robustez, D = mejoras) y referencia a archivo y línea. Al final hay una tabla de prioridades.

Lo que está bien (contexto)

Antes de los problemas, vale decir que la base es sólida y mejor que el promedio de un MVP:

- Arquitectura limpia y consistente: views flacas → services (escritura, transaccional) → selectors (lectura scopeada por rol). Se cumple en todas las apps.
- Reglas de dominio bien pensadas y documentadas en el código (R1, R10, R11, RP1–RP7, cadena de prórrogas con vigencia calculada y nunca persistida).
- Constraints reales en la base (UNIQUE parcial para R1, UNIQUE de documento vigente) además de la validación amigable en el service.
- JWT con rotación y blacklist, throttling, manejador de errores JSON uniforme, OpenAPI como contrato, paginación con tope.
- Tests de API en todas las apps; seeds reproducibles; docker compose con healthcheck.
- El pipeline del frontend (design intake → `build.py` con anclas que fallan ruidosamente → `dist/`) es ingenioso y está bien protegido contra pisadas.

A. Seguridad

A1 — Credenciales de admin hardcodeadas en el frontend ●

`frontend/integration/ceibo-api.js:14–18`

```
var CONFIG = { API: "...", USER: "admin", PASS: "Clave-Segura-123" };
```

Cualquiera que abra el front (o el repo) tiene la clave de un superusuario, y la app no tiene pantalla de login: todo visitante ES admin. Todo el sistema de roles del backend (Admin/RRHH/Supervisor/Empleado) queda sin efecto en la práctica. Es un atajo de MVP conocido, pero es **lo primero** a resolver antes de cualquier deploy o de dar acceso a terceros. Además la clave ya quedó en el historial de git: cuando exista un entorno real, rotarla.

Fix: pantalla de login que pida usuario/clave contra `/auth/token/` y guarde el refresh en memoria (o `sessionStorage`), y eliminar `USER / PASS` de `CONFIG`.

A2 — Fuga de scope: cualquier autenticado ve documentos de cualquier empleado ●

`gestion_rrhh/apps/empleados/api/views.py:73-78`

La acción `documentos` (GET) tiene permiso `IsAuthenticated`, pero resuelve el empleado con `get_object_or_404(Empleado, pk=pk)` **sin pasar por el selector**. El scoping por rol ("el Empleado ve solo su ficha") se aplica en `list / retrieve` vía `get_queryset()`, pero acá se lo saltea: un usuario con rol Empleado puede pedir `GET /api/v1/empleados/{cualquier_id}/documentos/` y ver los documentos (número, vencimientos, observaciones) de cualquier otra persona.

Fix: `empleado = get_object_or_404(self.get_queryset(), pk=pk)` en la rama GET (las acciones de escritura ya exigen RRHH/Admin, ahí no hay fuga).

A3 — PII expuesta a roles que no deberían verla ●

`gestion_rrhh/apps/empleados/api/serializers.py:36-65` vs. la promesa de `gestion_rrhh/apps/empleados/models.py:52` ("El PII (dni/cuil) se expone solo a RRHH/Admin").

`EmpleadoSerializer` es único para todos los roles: un **Supervisor** (que ve toda la dotación) recibe dni, cuil, dirección, teléfono, contacto de emergencia, obra social y observaciones de todos. El modelo documenta otra intención.

Fix: serializer reducido (sin PII) para Supervisor, o campos condicionales por rol en `to_representation`.

A4 — `SECRET_KEY` con default inseguro alcanza también a prod ●

`gestion_rrhh/config/settings/base.py:17` y `prod.py`

`SECRET_KEY = env("SECRET_KEY", default="insegura-solo-para-dev")` vive en `base.py` y `prod.py` no lo redefine: si en prod falta la variable de entorno, el proceso **arranca igual** con la clave insegura (firmando JWTs con ella).

Fix: en `prod.py`, `SECRET_KEY = env("SECRET_KEY")` sin default, para que prod explote al arrancar si falta.

B. Correctitud e integridad de datos resuelta (2026-07-15)

Los seis hallazgos (B1–B6) están arreglados y verificados contra Postgres. Se quitan de este documento; quedan acá solo el registro de qué se hizo y las consecuencias que sobreviven al fix. Detalle en el commit y en los tests que los cubren.

I D	Era	Quedó
B 1	Filtro empresa+estado usaba un JOIN por <code>.filter()</code> y cada condición la satisfacía una relación distinta	<code>empresa / sector / estado</code> se combinan en un solo <code>.filter()</code> — <code>empleados/selectors.py</code>
B 2	El PATCH podía romper la cadena de prórrogas (<code>fecha_desde</code> , <code>tipo_novedad</code>)	<code>actualizar_novedad</code> los rechaza si <code>novedad.es_prorroga</code>
B 3	El no-solapamiento solo vivía en Python (SELECT+INSERT, carrera abierta)	Lock pesimista sobre el empleado + <code>ExclusionConstraint</code> con <code>btree_gist</code>
B 4	Un documento cargado no se podía corregir ni renovar	PATCH / DELETE de <code>/empleados/{id}/documentos/{doc_id}/</code>
B 5	El legajo lo calculaba el navegador con <code>max+1</code>	Lo asigna el backend con advisory lock; el cliente no puede elegirlo
B 6	<code>todayISO()</code> devolvía la fecha UTC (de noche, mañana)	Se arma con getters locales

Un bug extra, no detectado en el análisis original: prorrogar una cadena que ya tenía una prórroga sin aprobar creaba **dos eslabones solapados arrancando el mismo día** — `vigencia_efectiva` solo avanza con las APROBADAS, y la validación de solapamiento no lo veía porque excluye la cadena entera. Ahora la cadena avanza de a un eslabón resuelto por vez. Sin este fix, B3 hacía explotar ese flujo con un `IntegrityError`.

Consecuencias que quedan vivas:

- **El fallback a sqlite murió:** `DATABASE_URL` es obligatoria y las migraciones solo corren en Postgres (el `ExclusionConstraint` no existe en sqlite). Ya era la regla de `CLAUDE.md`, pero antes el código la toleraba.
- `Novedad.ocupa_periodo` **es un dato denormalizado** (copia del flag del tipo, mantenida en `save()`), porque un `ExclusionConstraint` no puede hacer JOIN a `TipoNovedad`. Si algún día se cambia `ocupa_periodo` en un `TipoNovedad` ya usado, las novedades viejas conservan el valor con el que se cargaron: hay que backfilllear a mano.
- **Los locks de concurrencia no tienen test** (una carrera real no es testeable con la infra actual). Para novedades no importa demasiado: el `ExclusionConstraint` es la garantía dura y sí está testeado. Para el legajo no hay red equivalente — si el advisory lock fallara, el UNIQUE tira el mismo error críptico de antes.
- **B4 quedó deliberadamente mínimo:** sin flag `vigente` ni historial de versiones. Renovar un apto médico es mover su `fecha_vencimiento`. La decisión de versionar espera al módulo de documentos con archivos (Drive/OneDrive + metadata en base).

C. Robustez y rendimiento resuelta (2026-07-15)

Los cinco hallazgos (C1–C5) están arreglados y verificados contra Postgres (112 tests en verde + medición de queries sobre la base real + la app corriendo en el navegador). Se quitan de este documento; queda acá el registro de qué se hizo y las consecuencias que sobreviven al fix.

I D	Era	Quedó
C 1	<code>activo</code> hacía <code>.filter().exists()</code> , que ignora el prefetch: una query por empleado listado	Se resuelve sobre <code>relaciones.all()</code> — serializar 12 empleados pasó de 12 queries a 0
C 2	La serie de rotación disparaba 4 COUNT por mes; el panel abría con 71 queries (este documento decía ~50: estaba corto, medido son 71)	<code>_Dotacion</code> lee las relaciones una vez y cuenta en memoria — 71 → 5 queries
C 3	<code>getAllPages</code> no miraba <code>r.ok</code> : un 401/429/500 devolvía lista parcial y la UI decía "0 empleados" sin error	Lanza error como <code>jget</code> ; <code>page_size</code> 200 → 100 (el tope real del backend, antes se clampeaba en silencio)
C 4	Login y refresh compartían el scope <code>login</code> (5/min): varias pestañas renovando cortaban sesiones válidas	Scope <code>refresh</code> propio a 30/min — verificado: 8 refresh seguidos no gastan el cupo del login
C 5	El no-solapamiento traía todas las novedades del empleado y descartaba por fecha en Python	El rango se filtra en SQL, sobre el índice GiST que ya trae el <code>ExclusionConstraint</code>

Consecuencias que quedan vivas:

- **El dashboard ahora carga la dotación en memoria.** `_Dotacion` trae `(empleado_id, estado, fecha_ingreso, fecha_egreso)` de todas las relaciones. Con miles de filas son unas pocas tuplas; en **decenas de miles** habría que volver a agregación en SQL. El umbral está documentado en el docstring de la clase, que es donde se va a leer.
- **Cuidado con la asimetría al tocar** `_Dotacion: activos_*` cuenta **personas** distintas, `ingresos_en / egresos_en` cuentan **relaciones** (dos altas de la misma persona son dos ingresos). Es la semántica que ya tenían las queries, y es lo primero que se rompe al reescribir. La red es `test_dashboard_api.py`, que fija `hoy` y afirma números exactos.
- **Efecto lateral a favor:** las 71 queries eran 71 snapshots, así que una escritura concurrente a mitad de render podía dejar el KPI de activos peleado con la serie de rotación. Una sola lectura es un solo snapshot y eso desapareció.
- **C3 cambia el comportamiento a propósito:** donde antes había una lista a medias sin aviso, ahora hay una excepción. Es lo buscado (un fallo tiene que romper ruidoso, no mentir), pero es un cambio real: un endpoint caído ahora se ve.
- `_se_solapan` **ya no existe**. El predicado vivía en Python y ahora es `_filtro_solapadas_con` (un `Q`), para no tener la misma regla escrita dos veces y que se desincronicen. Se traduce 1:1, incluido el rango abierto (`fecha_hasta NULL`), que necesita `isnull=True` explícito porque en SQL `fecha_hasta >= x` con `NULL` no matchea.

- **Los throttles no tienen test automatizado:** C4 se verificó a mano contra la API viva (8 refresh → 401 sin 429; el 6º login fallido → 429). Un scope mal escrito explota al arrancar, así que el riesgo de que se rompa en silencio es bajo, pero un test de 429 seguiría siendo mejor que esta nota.

D. Mejoras funcionales y de proceso

D1 — Workflow de novedades incompleto

`EN_PROCESO` y `CERRADA` existen en el enum (`novedades/models.py:15-23`) pero no tienen endpoint de transición; el front los mapea a "sin acción" (`ceibo-api.js:152-154`). O se implementan (`/tomar/` , `/cerrar/`) o se quitan del contrato hasta que existan.

D2 — Sin auditoría (RP8)

El TODO está declarado en `novedades/services.py:7-9` : no hay `RegistroAuditoria` . Hoy solo se sabe quién creó (`creado_por`) y quién aprobó (`aprobada_por`); rechazos, anulaciones, ediciones y bajas no registran actor ni momento (el motivo se concatena en `observaciones` , que es frágil). Para RRHH — dominio con disputas legales— es de las mejoras de más valor.

D3 — Alertas de vencimiento ya estaba resuelto (constatado 2026-07-16)

El hallazgo era falso: existe `apps/dashboard/vencimientos.py` con sus tests (`test_vencimientos.py` , `test_alertas_dia.py`), el endpoint de por-vencer y el panel del dashboard cableado en `ceibo-api.js` . Ni `fecha_vencimiento` ni `Parametro` ni `referente_rrhh` están muertos.

D4 — Adjuntos ya estaba resuelto (constatado 2026-07-16)

El hallazgo era falso: hay `FileField` en `DocumentoEmpleado` (`empleados/models.py:234` , migración `0002_documentoempleado_archivo`) y en `AdjuntoNovedad` (`novedades/models.py:240`). Los archivos viven hoy en un volumen de media propio (`docker-compose.yml`), y el modelo ya prevé el pase a S3/R2 vía `STORAGES` sin tocarlo.

D5 — El front carga todo en memoria

`listEmpleados` / `listNovedades` traen **todas** las páginas y filtran client-side. Con cientos de registros va bien; con miles, la carga inicial y el ranking van a doler. El backend ya soporta filtros y paginación — el front podría usarlos cuando duela (no antes).

D6 — `getOrCreatePuesto` crea catálogo por texto libre resuelta en backend (2026-07-16)

El tipo seguía siendo texto libre en el canvas, así que el arreglo se puso donde la garantía no depende del cliente: `Puesto` tiene ahora unicidad **case-insensitive** por constraint (`UniqueConstraint(Lower("nombre"))`), el nombre se normaliza (`strip`) al guardar, y `POST /puestos/` es

idempotente: si el puesto ya existe devuelve el existente (200) en vez de crear un duplicado. "Chofer" / "chofer" / " CHOFER " colapsan en una sola fila. Migración en dos pasos (0003 fusiona los duplicados que ya existan y repunta sus relaciones; 0004 agrega el constraint) — van separadas porque Postgres no crea el índice único en la misma transacción que tocó filas con FKs.

Queda pendiente (UI): "Choferr" sigue creando un puesto nuevo. Eso lo cierra el `<select>` con "crear nuevo..." explícito, que es cambio de canvas (Claude Design) y no se hizo acá.

D7 — CI resuelta (2026-07-16)

El hallazgo estaba mal relevado: **sí existía** `.github/workflows/backend-ci.yml` (ruff + migrate + pytest contra Postgres de servicio). Se buscó en `Gestion Vial Victoria/` y no en la raíz del repo, que es donde vive `.github/`. Lo que faltaba de verdad se agregó:

- `frontend-ci.yml`: corre `python frontend/build.py`, que falla ruidosamente si un export de Claude Design rompe un ancla del cableado. Era la pieza que el análisis pedía.
- `backend-ci.yml`: el check de migraciones era `makemigrations --check --dry-run || makemigrations`. El `||` lo anulaba: en vez de fallar, generaba las migraciones al vuelo y seguía en verde. Ahora es un check estricto y sobre todas las apps (antes solo `usuarios` y `organizacion`).

D8 — Detalles menores del front resuelta (2026-07-16)

- `anios()` (`ceibo-api.js:132`): **arreglado**. Bajo el año informa en meses (y bajo el mes, en días), con singular/plural correcto. Además nunca dice "12 meses": a 364 días redondear al año falsea justo el dato que dispara antigüedad → tope en "11 meses".
- Novedades filtradas por empresa (`novedades/selectors.py`): **arreglado**. Cuando la novedad no trae `relacion_laboral`, la empresa se resuelve por las relaciones del empleado (el mismo criterio que ya usaba el ranking del dashboard), con `distinct()` para no duplicar a quien tiene relación en las dos empresas del grupo. Antes esas novedades no aparecían bajo **ninguna** empresa.
- `splitName()` (`ceibo-api.js:127`): **sin cambios, a propósito**. Es una heurística inevitable con un solo campo de nombre; separarlo en dos es decisión de canvas.

Prioridades sugeridas

Las secciones B y C salieron de esta tabla: están resueltas (2026-07-15). De D quedan D1, D2 y D5 (D3 y D4 nunca estuvieron pendientes; D6, D7 y D8 se cerraron el 2026-07-16). Lo que queda pendiente es **toda la sección A** y esas tres de D.

Prioridad	ID	Hallazgo	Esfuerzo
● 1	A1	Login real; sacar credenciales hardcodeadas	Medio
● 2	A2	Scope en GET /empleados/{id}/documentos/	Trivial (1 línea)
● 3	A3	PII solo para RRHH/Admin	Bajo
● 4	A4	SECRET_KEY sin default en prod	Trivial
● 5	D2	Auditoría (RP8)	Alto
● —	resto	D1, D5, D6 (el <select> del canvas)	según roadmap

Lo próximo, sin vueltas: la única sección roja es la de seguridad. A2 es una línea y hoy cualquier usuario con rol Empleado puede leer los documentos de cualquier otra persona. A1 es lo que vuelve real a todo el sistema de roles (hoy todo visitante es admin). Nada de eso se arregló acá: lo cerrado hasta ahora es robustez, rendimiento y las mejoras de proceso; seguridad sigue intacta.

Nota de método (2026-07-16): tres de los ocho hallazgos de D (D3, D4 y D7) estaban mal relevados — describían como faltante código que ya existía. Conviene verificar cada hallazgo contra el repo antes de planificar sobre esta tabla, y buscar desde la raíz del repo (`NEXOFLOWGESTIONVV/`), no desde `Gestion Vial Victoria/` : `.github/` vive un nivel más arriba y por eso se dio por inexistente.